

tickweave

Define, test, and run trading rules on FPGA.

System proposal and implementation source of truth

Document control	Value
Identifier / version	TW-SOT-001 / 0.1
Baseline date	October 2, 2026
Status	Initial implementation baseline; not a claim of completed work
Canonical format	tickweave_source_of_truth.md
Companion editions	Editable DOCX and matching PDF; same substantive requirements
Decision owner	Project lead; task owners review changes affecting their interfaces

Project proposition. tickweave is an OCaml/Hardcaml toolchain and operator console for expressing small trading rules, testing their meaning, compiling them into FPGA logic, configuring declared parameters, and inspecting the resulting hardware decisions. The demonstration uses an Arty-based Ethernet pipeline. Optional AI assistance translates researcher intent into validated proposals; optional voice dictation feeds the same text interface.

Required demonstration. Ethernet replay enters a hardware feed parser and one-instrument book. A generated rule evaluates a committed snapshot, passes through fixed admission checks, and emits a demo order to a host receiver. The terminal console shows the evidence and applies a reviewed parameter change through a cold-path interface.

Not a profitability claim. The project demonstrates implementation, configuration, verification, and observability. It does not establish trading profitability, production exchange compatibility, or production-grade risk management.

Current starting point. The project lead reports an existing feed parser and networking library. The order book, rule engine, order-submission engine, TUI, and supporting host workflow are not yet implemented. Existing code has not been audited by this document, and the exact board integration still requires bring-up.

Authority. This document consolidates the planning discussion into one baseline. Requirements below are project design choices, not claims that the software already exists. Examples and proposed commands are illustrative until implemented. Unresolved implementation choices are listed explicitly in Section 18 rather than silently fixed by an agent.

Reading guide

Audience	Start here
Proposal reviewer	1. Scope; 2. Architecture; 11. Ownership; 16. Delivery gates
FPGA implementer	3-7. Semantics, contracts, and control; 12-13. Hardware packages
TUI implementer	5. Contracts; 8-9. Console and evidence; 14. UI packages
Software / agent implementer	4-5. Rules and contracts; 10. Assistant; 15 and 17. Workflow
Whole team	16. Acceptance; 18. Open decisions and change control

Contents

1. Scope and decision baseline
2. System architecture and boundaries
3. Market state and order semantics
4. Rule language and hardware generation
5. Shared contracts and identity
6. Board communication and registers
7. Configuration lifecycle and failure behavior
8. Operator console and interaction design
9. Decision history, telemetry, and evidence
10. AI assistance and voice input
11. Team ownership and integration seams
12. Hardware work packages: H1-H5
13. Communication work packages: E1-E4
14. Console work packages: U1-U5
15. Software work packages: S1-S5
16. Delivery gates and demonstration acceptance
17. Extension tickets and CLI-agent working agreement
18. Open decisions, risks, and change control
19. Implementation references

Requirement language

MUST / MUST NOT identify required behavior within the stated priority. **SHOULD** identifies the preferred implementation; a departure needs a recorded rationale. **MAY** identifies an optional implementation choice, not new scope.

P0 — Required baseline: needed for the complete hardware/operator demonstration. **P1 — Staged extension:** useful only after the P0 path is reliable. **P2 — Deferred:** not part of this hackathon baseline. Priority does not imply implementation status.

How to update this document

Edit the Markdown first, record the affected decision or interface, increment the version, and regenerate the DOCX/PDF pair. Do not let a proposal edit, a chat message, or an agent's assumption silently become a competing specification. The team can deliberately revise this baseline as implementation reveals constraints.

1. Scope and decision baseline

1.1 Selected P0 architecture

Decision	Baseline
Identity	Lowercase <code>tickweave</code> ; SignalForge/SignalForce and RiskGate are historical brainstorming labels.
Target	Arty demonstration; no Alveo dependency. Exact board variant and resources remain to be confirmed.
Hardware path	Existing networking/parser, new one-instrument book, generated rule logic, fixed checks, and new order submission.
Rule model	Compile structure into the image; expose explicitly declared runtime parameters. No instruction interpreter.
Initial scale	One instrument and one rule slot. Carry identifiers so later expansion does not require a new conceptual model.
Control path	UART is the preferred cold-path transport. JTAG-to-AXI is an allowed bring-up substitute, not a second required implementation.
CPU	No RISC-V or other softcore is required or planned.
Application	OCaml-based interactive TUI; <code>bonsai_term</code> is the intended stack, pending a pinned working environment.
Verification	Separate software evaluator, hardware simulation, fixtures, and an actual hardware demonstration.

1.2 Feature priorities

P0 includes a small declarative rule representation; validation and software evaluation; Hardcaml generation; a versioned manifest/profile; basic register control; reviewed configuration application; rule/market/status views; inspectable decision history; a local demo-order receiver; and explicit mock, simulation, and hardware modes.

P1 includes an assistant pane and one model backend; natural-language-to-rule proposals using existing tools; ElevenLabs record-then-transcribe dictation; a second example strategy; and optional additional compiled slots. Each extension must preserve the P0 architecture and remain removable.

P2 includes a Bonsai web GUI, live arbitrary-rule interpretation, a softcore, fast parameter updates, shadow-bank hot swapping, sophisticated stateful indicators, a general expression language, automated strategy discovery, live trading, portfolio risk, full order lifecycle management, multiple AI backends, MCP, embedded terminal emulation, editor plugins, and continuous voice conversation.

1.3 Risk checks, not a second risk product

Fixed admission checks **MUST** precede order submission. They cover enable state, valid market state, supported quantity bounds, and any explicitly defined price bounds. They emit reasons for rejection. Position reservation, fills, working-order exposure, cancel/replace, and portfolio accounting are outside P0. The control is **block new evaluations/orders**, not a claim to cancel previously sent orders.

1.4 Scope-reduction order

Remove voice, then live assistant integration, extra slots/rules, and decorative visualizations before compromising the hardware path or its evidence. Synthetic-snapshot harnesses are valid bring-up tools but do not replace the required Ethernet-to-order demonstration. If a P0 requirement must change, record the change and its effect on the final claim.

2. System architecture and boundaries

2.1 FPGA data path

```
Host Ethernet replay
-> existing networking and feed parser
-> normalized market updates + commit boundary
-> one-instrument order book
-> committed market snapshot
-> DSL-generated rule circuit
-> fixed admission checks
-> order submission / packet encoder
-> host demo-order receiver
```

The host supplies feed data, not live trading decisions. Hardware **MUST** determine whether the compiled rule produces an order candidate. The host receiver is a demo endpoint, not a real exchange.

2.2 Host control and authoring path

```
OCaml declarations / structured proposal
-> validated rule representation
  -> independent software evaluator
  -> Hardcaml hardware generator
  -> manifest and deployment planner

TUI / CLI -> shared application controller
          -> one register transport -> FPGA configuration

FPGA telemetry + host order-receiver events
          -> application event model -> TUI and logs
```

Hardcaml provides circuit construction, simulation, and RTL generation; tickweave supplies the trading-specific representation, semantics, wrappers, and workflow. [1]

2.3 Required module boundaries

Rule modules consume typed snapshots/configuration and emit candidates plus metadata. They **MUST NOT** depend on UART, AXI addresses, terminal components, or AI providers.

The board wrapper owns reset/clocking, register access, engine state, and trace buffering. **The submission engine** owns candidate serialization and transmission, not rule evaluation. **The host controller** owns proposal validation and the apply sequence. **The TUI** renders application state and submits commands; it does not write registers directly.

The model adapter can call permitted application tools but does not own deployment. **The transcription adapter** produces editable text only. Cloud requests **MUST NOT** block hardware monitoring or control.

2.4 Process and repository shape

A shared host library inside the TUI process is sufficient initially. A separate service becomes appropriate only when multiple concurrent clients need the same board connection. Exactly one component owns the device connection and serializes requests.

Suggested modules are `tickweave_types`, `tickweave_rule`, `tickweave_eval`, `tickweave_hardcaml`, `tickweave_control`, `tickweave_transport`, and `tickweave_tui`, with fixtures and tests alongside them. These are organizational suggestions, not already existing packages. Keep shared contracts small and avoid one monolithic application file.

3. Market state and order semantics

3.1 Market-model boundary

P0 targets one instrument. The intended book is fixed-depth market-by-price, aligned with the existing parser's supported feed semantics. Before H1/H2 implementation, freeze the feed subset, retained depth, indexing convention, initialization sequence, and commit rule. The precise feed/schema version remains an open implementation item.

The strategy sees best bid/ask prices and quantities from a **committed snapshot**. A parsed field, incremental level update, packet boundary, and completed market event **MUST NOT** be assumed equivalent. The adapter/book contract defines when the snapshot is consistent enough to evaluate.

Retaining only the current best level is insufficient whenever supported deletions require revealing the next level. Tests must exercise that transition. A top-of-book snapshot fixture is allowed for isolated testing, but must be labelled as such rather than passed off as an incremental-book implementation.

3.2 Validity and recovery

The book starts invalid. Reset, detected sequence loss, malformed required updates, or unsupported state-changing events for the selected instrument **MUST** follow a documented invalidation policy and suppress candidates. Explicit reinitialization from a known replay sequence is sufficient; full feed recovery is deferred. Events for irrelevant instruments may be ignored only under the frozen feed contract.

No sequence-gap detection may be claimed unless the parser/adapter actually supplies and checks the necessary identifiers. Missing detection is a documented limitation, not proof of continuity.

3.3 Evaluation and action

Evaluate once per accepted committed snapshot while the engine is enabled and the book valid. A condition that remains true may fire again on a later snapshot; the initial language is level-triggered, not a crossing detector. It **MUST NOT** emit one order on every clock simply because a predicate stays true.

The first rule produces zero or one candidate. A candidate contains stable decision, rule, and configuration identifiers plus side, price, and quantity. Fixed checks admit or reject it before submission. If a future pack supports conflicting actions, conflict/arbitration behavior must be explicit; an initial multi-slot extension may use fixed priority and record matches that were not selected.

3.4 Submission and backpressure

The demo-order format **MUST** be versioned and include fields needed to correlate host receipt with the hardware candidate. Full exchange sessions, authentication, execution reports, fills, and cancellations are not required.

At every hardware stream boundary, define when an item is accepted. An item stalled by downstream readiness **MUST** retain stable data and identifiers, and may be accepted only once. Buffer depths and overload policies must be recorded. Telemetry must never silently backpressure the trading path; order-path congestion must have a deliberate, observable policy. P0 can use slow deterministic replay, but must still test stalls and loss conditions honestly.

4. Rule language and hardware generation

4.1 Shared representation

An OCaml declaration frontend constructs a small, typed rule representation. A structured-data frontend for CLI/AI proposals may construct the same representation. Avoid a custom text grammar until the semantics and evaluator work. JSON versus S-expression interchange is an interface-freeze decision, not a reason to implement two competing systems.

The evaluator and Hardcaml backend implement the agreed semantics independently. Sharing an AST and metadata is desirable; making the reference simply repeat the hardware output is not verification. Arbitrary user/model-generated OCaml is not executed as a rule payload.

4.2 Initial semantics

P0 needs field references, explicitly declared parameters, integer constants, and the comparison required by the first rule. Add Boolean combinations, addition/subtraction, and constant multiplication only as examples require and both backends support them. Every admitted operator needs validation, evaluation, generation, and boundary tests.

Prices are integer ticks; quantities are integer units. Widths, signedness, widening, legal ranges, subtraction behavior, and overflow policy **MUST** be frozen before S2/H3 are accepted. Do not inherit accidental host-integer behavior. Loops, floating point, division, arbitrary functions, moving windows, and user-defined state are deferred.

4.3 First required rule

```
Rule: buy_below_limit
Inputs: valid book snapshot; enabled configuration
Condition: ask_px <= maximum_price
Action: BUY quantity at ask_px
Trigger: once per accepted committed snapshot
Example defaults: maximum_price = 1005 ticks; quantity = 1
```

The example values are not investment advice or tuned strategy parameters. A second rule may add a minimum spread or an imbalance predicate after the first path works.

4.4 Build time versus runtime

Change	Deployment class
Change a declared parameter or enabled state	CONFIGURATION_ONLY
Change rule structure supported by the compiler but not the loaded image	REBUILD_REQUIRED
Request an unsupported operation or invalid semantics	UNSUPPORTED

A **rule definition** describes one computation and action. A **strategy pack** describes compiled rules, slots, and arbitration. A **runtime profile** holds permitted parameter values and enabled states. An **FPGA image** is the built bitstream. Use these terms consistently.

The build produces a manifest describing the actual image's capabilities. The planner must distinguish language support from what is physically instantiated. Parameters feed the compiled circuit as data; they do not rewire it. No arbitrary-rule hot-loading claim is permitted.

5. Shared contracts and identity

5.1 Interface package

The project lead owns the first shared `.mli` contracts and hand-checked fixtures. Each owner reviews changes that cross their boundary. These are logical fields, not a finalized byte layout.

Contract	Required contents / meaning
Market update	Selected instrument, supported operation, level/price/quantity fields, source sequence context, and commit information.
Market snapshot	Instrument; snapshot ID; bid/ask prices and quantities; validity; source context needed for replay correlation.
Rule definition	Rule ID/name, typed predicate/action, declared parameters, and supported trigger semantics.
Configuration	Parameter values, enabled state, and configuration version; distinguish requested from acknowledged active values.
Candidate	Run/decision identity, rule ID, configuration version, side, price, and quantity.
System event	Schema version, mode/source, run ID, event sequence, event type, relevant decision/rule/configuration IDs, and payload.
Manifest	Build identity, register/API schema identity, compiled rule capabilities, parameter types/units/ranges, and register mapping.
Proposal	Target build, base configuration version, normalized change, deployment class, validation/test evidence, and proposal ID.

5.2 Identity rules

A **run ID** changes across resets/replay sessions so reused counters cannot be confused. A **snapshot ID** identifies the committed input. A **decision ID** identifies an evaluation and travels with its candidate and related outcomes. A **configuration version** identifies the exact configuration used throughout that evaluation.

The mapping between snapshots and decisions must be explicit when disabled periods skip evaluations. Counter widths, wrap behavior, and reset reconciliation belong in the binary/schema contract. A host timestamp is not a substitute for hardware event identity.

5.3 Event vocabulary

Keep `rule_matched`, `candidate_created`, `candidate_admitted`, `candidate_blocked`, `order_transmitted`, and `order_received` distinct. Also represent connection/reset, configuration progress/failure, book validity, and dropped telemetry. A design may combine wire records, but the application must preserve these semantic distinctions.

5.4 Command and fixture boundary

Frontends submit application commands such as inspect decision, validate proposal, apply proposal, or block new evaluations. They do not assemble register addresses. Fake and real backends expose the same logical command/event interfaces.

Supply deterministic fixtures for connected/disarmed, enabled, no signal, admitted candidate, blocked candidate, received order, update pending/failed, and connection lost. Include provenance and expected outcomes. Mock, simulation, and hardware modes **MUST** remain visibly distinguishable in the console and exported evidence.

6. Board communication and registers

6.1 Transport selection

Ethernet carries market replay and outbound demo orders. UART is the preferred independent cold path for parameters, enable/disable control, status, and bounded telemetry. Fast field changes are outside this baseline. No host memory mapping or PCIe dependency is required: the host issues logical register operations over a transport.

A UART implementation consists of byte RX/TX, framed command decoding, register access, and responses. These are hardware state machines and registers; a softcore is not necessary. The FPGA register interface may be a small custom request/response bus. AXI is optional rather than an application-wide requirement.

JTAG-to-AXI is an allowed bring-up substitute if it reduces integration risk. Keep any Tcl adapter limited to register operations behind the OCaml transport API. Do not put rule semantics or deployment policy in Tcl. The exact vendor setup must be verified against the chosen tool version before adopting this fallback.

Service-bus-over-UDP is a later alternative, not P0. It would need a response path, transaction tracking, ordering/duplicate handling, and congestion policy. A separate destination port alone does not supply those properties. Do not implement several transports solely to demonstrate abstraction.

6.2 Minimum register protocol

Begin with logical 32-bit reads/writes and one outstanding host request. Freeze the framing, protocol version, byte order, address alignment, length limits, transaction identifier, checksum/CRC, response codes, and timeout behavior before E3 integration. Checksum choice and register addresses remain open in Section 18.

A successful write response means the register operation was accepted, not merely that bytes arrived. Invalid addresses, unsupported versions, malformed frames, and illegal writes **MUST** produce defined failures or bounded resynchronization. Stateful commands such as activation require idempotent transaction handling or status reconciliation; do not blindly repeat them after an ambiguous timeout.

6.3 Register-bank requirements

Expose build/schema identity, scratch read/write, engine enable/idle status, active configuration version, parameter registers, and relevant counters/faults. Generate hardware decoding, host accessors, and manifest information from one definition where practical. The UI and assistant consume names/types, not raw addresses.

Parameters **MUST NOT** be modified while the engine may still consume them. Enforce or explicitly reject unsafe writes at the register wrapper rather than relying only on a cooperative UI. Reset begins disarmed. The engine clock domain should own configuration where practical; asynchronous UART inputs and any real clock-domain crossings require explicit synchronization/CDC handling.

6.4 Control versus trace traffic

Command responses take priority over telemetry. Trace buffering is bounded and exposes loss counters. The transport **MUST** remain responsive when trace production exceeds serial bandwidth. A working feed-receive library is not evidence that the required transmit path is already validated.

7. Configuration lifecycle and failure behavior

7.1 One authoritative apply operation

A person reviews a concrete proposal against the connected build and current configuration version. The host controller serializes the following operation; TUI, CLI, and assistant integrations **MUST NOT** reimplement it independently.

```
Validate target build, base version, fields, and ranges
-> request disable / block new evaluations
-> wait for DISABLED_AND_IDLE acknowledgment
-> write permitted parameter values
-> read back and verify
-> activate the complete configuration with a new version
-> read/acknowledge active state and version
-> report success to the application
```

A standalone enable-bit write is not proof of idle. Existing evaluations may still read parameters in later stages. Idle must cover all consumers of mutable configuration. A pipeline-valid reduction or an outstanding-work counter is an implementation option, not a requirement to use one specific circuit.

7.2 In-flight policy

The baseline allows already completed candidates to finish submission with their original configuration tag. Disabling prevents new evaluations, waits for existing rule evaluations to complete, and does not retroactively cancel packets or orders. Candidates **MUST** contain all values needed downstream; the submission engine must not read mutable rule parameters later.

The feed parser and book may continue updating while a rule is disabled. Re-enabling evaluates future committed snapshots, not a backlog of missed events unless an explicitly revised contract says otherwise. For the simplest demo, replay may be paused during updates, but the controller must still implement the acknowledged lifecycle.

7.3 Failure and uncertainty

If validation, idle wait, parameter writes, or readback fails, do not issue activation. When communication remains available, keep/confirm the engine disarmed and surface the error. Never automatically re-enable a partially updated configuration.

If communication is lost, the host may not know whether the last operation took effect. Report **state unknown** with the last acknowledged version, not an unverified claim that hardware is disabled. In particular, a lost activation acknowledgment requires status reconciliation before further action. A host disconnect does not imply an automatic hardware kill; a watchdog would be a separately designed feature.

Reset invalidates the previous runtime session. Reconnect must read build identity, reset/run context, state, and configuration before enabling controls. A proposal against a different image or stale base configuration **MUST** be rejected or replanned. UI state distinguishes draft, validated, applying, acknowledged active, failed, and unknown.

7.4 Structural deployment

Rule-source changes require validation, evaluation, hardware generation, synthesis/implementation, programming, and a matching manifest. Do not auto-program on editor save. Record source/build provenance and begin the new image disarmed. Shadow registers, uninterrupted hot swapping, and partial reconfiguration are outside P0.

8. Operator console and interaction design

8.1 Application identity

The TUI is an interactive operator console, not merely a log viewer and not an embedded IDE. `bonsai_term` is the intended implementation, with `bonsai_term_examples` for component patterns and `btop` for compact, readable interaction design. The current Bonsai terminal documentation uses `OxCaml`; pin a tested environment before parallel frontend work. [2-6]

Area	Required presentation / action
Status header	Mode, connection, build/run identity, active version, armed/idle/unknown state, book validity, and trace-loss status.
Market panel	Current best bid/ask and quantities with explicit units and validity.
Rules panel	Compiled rules, enabled state, parameter values, counters, and selected rule.
Decision history	Stable event selection, filtering, scrolling, and a detail view tied to the selected record.
Configuration review	Old versus proposed values, target build/base version, validation results, and explicit Apply.
Assistant pane (P1)	Hideable conversation, editable draft, tool progress, cancel action, and links to structured proposals.

8.2 Interaction requirements

New events **MUST NOT** silently replace the inspected decision or force scrollbar to the bottom. Keep selection by stable identifiers rather than row offsets. Hiding a pane preserves its state. Text-input focus must prevent dashboard shortcuts from interpreting normal typing as commands.

Support a usable single-terminal layout first, responsive resize behavior, visible keyboard help, and compact error/status text. Multiple monitors are a presentation convenience, not a deployment requirement. Status must not depend only on color. Never show an optimistic active configuration before hardware acknowledgment.

The frontend consumes application state/events and emits commands. Each panel should be independently testable with fixtures. Ordinary controls must remain usable during a model request, transcription, slow simulation, or transport timeout. Background worker execution may be used inside the implemented application; the UI event loop must not block on those operations.

8.3 Reference mapping

Inspect `hello_world`, `responsive_dimensions`, and `typography` for the shell; `events` and `click_handler` for interaction; `scroller` for history; and `text_box/text_editor` for editable controls. [3] Reusable border boxes, scrollers, editors, spinners, charts, and themes are available in the components repository. [4] Terminal expect-test support is available in `bonsai_term_test`. [5]

`btop`'s selection, filtering, sorting, detail views, list pausing, and keyboard/mouse controls are relevant interaction references. They are not a request to copy its system-monitor feature set or assets. [6] Prefer readable tables, restrained borders, explicit units, and useful event details over decorative graphs.

A separate Bonsai web frontend, waveform/schematic browser, graphical rule editor, and a literal nested terminal application are deferred.

9. Decision history, telemetry, and evidence

9.1 Explain a particular decision

The core question is: **which inputs, which rule, and which active configuration produced this outcome?** A detail view must preserve those values even while the live book changes.

```
Run: demo-03           Decision: 417
Build: <recorded ID>    Configuration: 12
Rule: slot 0 / buy_below_limit
Ask: 1002 ticks        Maximum price: 1005 ticks
Predicate: true        Candidate: BUY 1 at 1002
Admission: accepted    Order received: yes
```

This is an illustrative record, not a measured result. The implementation stores/attaches the actual decision inputs and configuration. The software evaluator may reconstruct a richer explanation, but reconstructed values must be labelled separately from hardware-recorded fields.

9.2 Trace isolation

A bounded hardware trace queue feeds the host. Overflow increments a visible counter and may drop/summarize trace data; it **MUST NOT** silently stall rule evaluation or order submission. Host history is also bounded or persisted with a documented retention policy. A quiet console does not prove no events occurred when loss is reported.

P0 may replay slowly enough to capture complete histories. Freeze trace size, UART rate, queue depth, and replay pacing before claiming loss-free capture. The design need not record every no-signal update at full rate, but any completeness claim must match the configured capture policy.

9.3 Layered verification

Level	What it establishes
Parser / adapter fixtures	Known packets produce correct normalized updates and commit markers.
Book fixtures	Update sequences produce correct snapshots, best-level transitions, and validity.
Evaluator / hardware comparison	The generated rule agrees with independently checked semantics on identical snapshots/configurations.
Submission tests	An accepted candidate is encoded and received without field corruption or duplicate acceptance.
Controller / TUI tests	Requests, errors, acknowledgments, and state transitions are represented correctly.
Board demonstration	The integrated FPGA actually consumes replay and emits correlated demo orders.

Hand-check representative expected results before treating the evaluator as an oracle. Sharing input fixtures is appropriate; silently generating every expected value with the same logic under test is not.

9.4 Measurement and reporting

Record the implemented clock and timing outcome, then measure latency between named events such as snapshot acceptance and candidate availability. Report stalls or give a distribution when latency is variable. Keep this separate from Ethernet serialization, UART latency, host receipt time, and end-to-end round trips. No fixed nanosecond, line-rate, or resource-utilization target has been agreed.

The demo evidence must state mode, source/build version, replay identity, parameters, input/output counts, mismatches, and drops. Simulation does not establish timing closure; hardware/reference agreement does not establish a profitable strategy or a correct book unless those layers were separately tested.

10. AI assistance and voice input

10.1 P1 researcher workflow

The assistant translates an idea into a supported rule or parameter proposal, runs existing validation/tests, and explains the returned evidence. It does not invent a new hardware capability or become the trading decision path. No model training, on-FPGA AI, or profitability-discovery claim is planned.

```
Typed / transcribed request
-> normalized rule proposal
-> validate -> simulate -> plan deployment
-> operator review -> normal controller applies
```

A native assistant pane with one model API backend is preferred. Standard tool calling lets the model request operations that application code executes. [7] An external CLI agent may exercise the same tools during bring-up. Actual Codex App Server integration, MCP, multiple backends, and literal terminal embedding are deferred rather than additional P1 obligations.

10.2 Tools and authority

Expose bounded operations equivalent to `describe_capabilities`, `validate_rule`, `simulate_rule`, `plan_deployment`, and `inspect_decision`. Tool results contain structured errors and evidence identifiers. The planner reports `CONFIGURATION_ONLY`, `REBUILD_REQUIRED`, or `UNSUPPORTED` using the connected manifest, not model intuition.

Do not expose raw registers or unrestricted activation to the model. Enforcement belongs in application permissions, not only a prompt. Structured proposals hold the target build, starting version, changes, validation, and tests; the review UI reads those objects rather than parsing conversational claims such as "tests passed."

Resolve ambiguous units, inclusive/exclusive boundaries, and level-triggered versus crossing semantics before approval. Attach selected rule/decision context explicitly and freeze it at message submission. Treat source files, transcripts, and logs as untrusted input; they cannot grant new tool permissions. Limit test execution and file access to intended resources.

10.3 Voice is input, not a separate agent

P1 voice uses ElevenLabs speech-to-text: record, stop, transcribe, insert editable text, then explicitly send. ElevenLabs exposes an audio-file transcription endpoint suitable for this bounded workflow. [8] Keyboard input remains available if recording or the service fails.

Capture audio on the local host, show recording status, and preserve user edits. No silence detector, completed transcript, or voice phrase automatically submits a prompt or deploys a configuration. Realtime transcription, speech output, wake words, and interruption handling are deferred.

10.4 Reliability and privacy

Keep provider credentials outside source control and client-visible logs. Send only the deliberately selected context; do not upload the full feed, private logs, or secrets by default. Document that recordings go to the transcription provider and selected prompts/context go to the model provider. Failures in either service must leave manual configuration, monitoring, and the deterministic demo usable.

11. Team ownership and integration seams

11.1 Four owners

Owner	Primary responsibility	Critical review boundary
Project lead	Shared semantics, feed/book, hardware generation, engine wrapper, and final FPGA integration.	Reviews all changes to widths, triggers, manifests, and hardware contracts.
Experienced teammate	Order encoding/transmission, host receiver, register transport, and telemetry adapter.	Reviews wire formats, CDC/handshakes, failure handling, and integration with the board.
Beginner A	TUI shell, market/rules/history panels, proposal review, and later assistant presentation.	Receives application data/actions; does not implement raw deployment transactions.
Beginner B	Fixtures, evaluator/validation, profiles/planning, replay/comparison, and host controller.	Lead reviews semantics; lead/experienced teammate reviews deployment lifecycle.

These are ownership roles, not named assignments. Each ticket remains separately testable and may be reassigned without changing architecture.

11.2 Initial handoff: C0 interface freeze

The lead supplies shared `.mli` contracts, the first rule, checked input/output examples, a fixture event stream, and a working Dune environment. Resolve the blockers in Section 18 needed for the first implementation. Do not require a complete general framework before this handoff.

The experienced teammate starts a synthetic-candidate-to-host transmission harness. Beginner A starts a selectable rules view against fixtures. Beginner B starts the first evaluator and fake backend. The lead can implement book tests and generated rule tests independently because their harnesses accept normalized updates and snapshots directly.

11.3 Dependency seams

Boundary	Producer -> consumer	Artifact to agree first
Market	Parser adapter -> book -> rule	Update/commit and snapshot records; validity semantics.
Orders	Rule wrapper -> submission -> host	Candidate handshake and versioned demo packet.
Configuration	Host controller -> transport -> wrapper	Read/write/status semantics and complete apply state machine.
Console	Backend -> TUI	Events, commands, stable IDs, and fake responses.
Assistant	Model adapter -> application tools	Structured proposal/results; no deployment bypass.

11.4 Assignment discipline

A ticket includes one owner, scope, input example, output example, acceptance test, dependency, and reviewer. Give beginners the next concrete deliverable rather than an open-ended subsystem. Shared interfaces require review; ordinary internal implementation does not need a team-wide redesign.

Use separate modules/panels and small changes to reduce merge conflicts. New shared dependencies and toolchain changes require lead coordination. Each handoff includes the actual test command and result, not only a screenshot or a claim that the component works.

12. Hardware work packages: H1-H5

Owner: project lead. **Priority:** P0. Each package has an isolated harness; integration is not its only test.

H1 — Feed-to-book adapter

Input/output: existing parser records become normalized market updates and explicit commit markers. Preserve sequence context and selected-instrument handling. **Acceptance:** supplied packet fixtures yield exact expected operations/commits, including malformed or unsupported cases. **Dependencies:** C0 feed contract and checked parser fixtures. **Handoff:** normalized update fixtures for H2 and S4. Do not rewrite the networking library or broaden protocol coverage without a recorded requirement.

H2 — One-instrument book

Input/output: normalized updates become valid committed best-bid/ask snapshots from the retained depth. **Acceptance:** add/change/delete sequences, best-level deletion, initialization/reset, invalidation, and boundary-depth cases match hand-checked snapshots. **Dependencies:** C0 update contract; H1 is not needed to run isolated tests. **Handoff:** snapshot fixtures and validity rules for H3/S2. No order-ID matching engine or general recovery system.

H3 — Rule hardware generator

Input/output: the validated rule representation produces a Hardcaml circuit with snapshot/configuration inputs and candidate/metadata outputs. **Acceptance:** the first rule and every supported operator match S2 on the same test cases; exact thresholds and arithmetic boundaries are covered. **Dependencies:** C0 semantics and representative fixtures; H2 may be replaced by a synthetic source in tests. **Handoff:** standalone generated-rule simulation, RTL output, and capability metadata. No runtime expression interpreter.

H4 — Engine wrapper and fixed checks

Input/output: snapshots, configuration writes, and control requests become gated evaluations, admitted/blocked candidates, status, and traces. Own decision/configuration IDs, disarm/idle, reset behavior, and stable candidate handoff. **Acceptance:** disabled/invalid state admits no new evaluations; unsafe writes are rejected; drain-before-update works; pending candidates retain their original values/version. **Dependencies:** C0 register/stream contracts and H3 harness. **Review:** experienced teammate checks register and submission boundaries.

H5 — Board integration and measurement

Input/output: combine parser, H1-H4, E2, and E3/E4 into the demonstration image. **Acceptance:** a known replay produces correlated host-received orders; configuration change affects later decisions; timing/clock and defined latency endpoints are recorded; reset and backpressure tests pass. **Dependencies:** G1 transport evidence and passing component tests. **Handoff:** reproducible build/program instructions, matching manifest, replay/profile, and captured results. Do not substitute software-made decisions for the hardware rule.

Parallel work rule

H2 and H3 can proceed independently after C0. H4 can begin against stub rule outputs and fake register requests. S4 supplies comparisons; the lead remains responsible for reviewing their expected semantics. Freeze successful component behavior before expanding rule count or adding richer arithmetic.

13. Communication work packages: E1-E4

Owner: experienced teammate. **Priority:** P0. Prove the outbound path and cold-path access before depending on them to debug trading behavior.

E1 — Demo-order format and host receiver

Input/output: a documented candidate schema becomes a versioned binary packet and a host-decoded event.

Acceptance: golden byte vectors round-trip all IDs, side, price, quantity, and version; truncated/unknown-format packets are rejected; receipt is distinguishable from exchange acceptance. **Dependencies:** C0 candidate contract. **Handoff:** packet layout, fixture bytes, host receiver, and application event adapter.

Addresses/ports/endianness must be frozen, not guessed independently by host and FPGA implementations.

E2 — FPGA order submission

Input/output: accepted candidates become outbound frames through the existing networking library.

Acceptance: a synthetic candidate is decoded unchanged at the host; repeated/stalled handshakes do not duplicate acceptance; output fields stay stable until accepted; queue/overflow behavior is tested and documented. **Dependencies:** E1 and the candidate/TX stream contract. **Handoff:** standalone on-board synthetic-order harness. No real exchange login, TCP session, fills, or cancel/replace requirement.

E3 — Cold-path register transport

Input/output: host read/write operations become acknowledged register requests on the board. UART is preferred; a documented JTAG substitute is allowed. **Acceptance:** read build/schema identity, write/read scratch, reject invalid addresses/frames, recover from truncation, and surface timeout/ambiguous outcomes.

Dependencies: C0 transport and H4 register contracts; a scratch-only bank is sufficient for initial bring-up.

Handoff: a narrow OCaml API plus a fake transport for S5. Do not implement deployment policy here.

E4 — Telemetry transport and host adapter

Input/output: hardware trace/status becomes the shared application event model; host order receipt is a separate source. **Acceptance:** IDs and versions survive transport; fragmented input is handled; control responses remain responsive under trace load; overflow/drop counts are surfaced. **Dependencies:** C0 event schema, E3, and H4 trace contract; fixture streams can stand in for hardware. **Handoff:** an event feed usable by S4 and the TUI. No frontend-specific binary parsing.

First success and review

The first success is **synthetic FPGA candidate -> decoded host order**, followed by **host request -> scratch register -> response**. This can be completed before the book or rules exist. The project lead reviews candidate/register semantics; the experienced teammate reviews H4/H5 integration at their boundary. A passing host codec test alone is not evidence that the FPGA transmitted anything.

14. Console work packages: U1-U5

Owner: Beginner A. **Review:** lead for semantics, Beginner B for application events. U1-U4 are P0; U5 is P1 and begins against fixtures.

U1 — Application shell

Input/output: sample application state becomes a resizable terminal layout with status header, panel focus, keyboard help, and explicit mode. **Acceptance:** build/run without board or credentials; resize and focus tests pass; controls do not overlap or silently disappear at the agreed minimum terminal size. **Dependencies:** C0 toolchain and S1 fixtures. Use the examples as patterns, not a mandate to re-create every demo component.

U2 — Market and rules panels

Input/output: current market/rule/configuration records become selectable views and application actions. **Acceptance:** units, valid/invalid state, enabled state, and active version are correct; selection uses IDs and survives updates. **Dependencies:** U1 and shared records. **Handoff:** isolated panel tests. Do not display draft values as active and do not encode register addresses in the UI.

U3 — Decision history and inspector

Input/output: events become a bounded, filterable history and a stable selected-decision detail view. **Acceptance:** selection/scrollback is not stolen by new events; missing data and drops are visible; recorded inputs and version remain attached to the historical decision. **Dependencies:** U1 and S1/E4 event interface. **Handoff:** deterministic interaction tests and a sample inspection of decision 417. Real hardware provenance is required before labelling data as hardware evidence.

U4 — Configuration review

Input/output: manifest, draft, and structured proposal become validated controls and an explicit apply command. **Acceptance:** old/new values and target build/version are visible; pending, active, failed, and unknown differ; stale proposals cannot silently apply; fake backend covers each response. **Dependencies:** S3/S5 interfaces, which may initially be fixtures. **Handoff:** UI-to-controller command integration. This panel does not sequence raw register writes.

U5 — Assistant pane

Input/output: conversation/tool/proposal events become a hideable transcript, editable input, and review actions. **Acceptance:** focus and scrollback are stable; hiding preserves state; cancel/errors are represented; normal monitor controls remain usable; proposal links refer to backend objects. **Dependencies:** U1/U3/U4 contracts; prerecorded events before X1. **Priority:** P1. No embedded shell or terminal emulator.

First assignment

Build U1 and a simple selectable U2 rules view from supplied data. Next, add U3. Do not assign model integration, microphone capture, and board control simultaneously. Keep each panel in its own module with a clear input/action interface.

15. Software work packages: S1-S5

Owner: Beginner B. **Priority:** P0. The project lead reviews evaluator semantics; lead/experienced teammate reviews the controller before it controls hardware.

S1 — Fixtures and fake backend

Input/output: reviewed scenarios generate deterministic application events and fake command responses.

Acceptance: connected/disarmed, enabled, no signal, admitted, blocked, received, update pending/failed, and disconnect/unknown cases are reproducible and visibly mock. **Dependencies:** C0 examples. **Handoff:** event files and fake backend that unblock U1-U4. Expected hardware results must not be fabricated for presentation.

S2 — Rule loading, validation, and evaluation

Input/output: agreed structured rules/profiles and snapshots become normalized rules, useful errors, and expected decisions. **Acceptance:** first-rule equality boundary, true/false, invalid fields/ranges, and every supported arithmetic operator match reviewed examples. **Dependencies:** C0 typed semantics. **Handoff:** reusable library plus machine-readable CLI results for H3/S4/X1. Avoid a custom parser and never treat host integer overflow as the specification.

S3 — Manifest, profile, and proposal handling

Input/output: connected capabilities plus requested changes become a versioned profile/proposal and CONFIGURATION_ONLY, REBUILD_REQUIRED, or UNSUPPORTED classification. **Acceptance:** reject missing/invalid parameters and mismatched builds/versions; persist exact normalized changes and test references; distinguish compiler support from loaded circuits. **Dependencies:** C0 manifest and S2 validation. **Handoff:** application objects for U4/S5; not prose that the UI must interpret.

S4 — Replay and comparison runner

Input/output: verified packets/snapshots, a profile, and observed events become paced replay and correlated comparison reports. **Acceptance:** match by run/snapshot/decision/configuration identity, report exact mismatches and missing/extra outcomes, and separate book tests from same-snapshot rule tests. **Dependencies:** S1/S2 and lead-supplied feed fixtures; E1/E4 for board evidence. **Handoff:** deterministic scenario/test command and machine-readable report. Do not reverse-engineer the feed independently.

S5 — Host configuration controller

Input/output: an approved proposal and transport API become the complete disable/idle/write/verify/activate transaction and application status. **Acceptance:** fake-transport tests cover every failure point, stale versions, reconnect, and ambiguous activation response; no activation after failed verification; unknown state is not reported as confirmed disabled. **Dependencies:** S3, E3 interface, and H4 lifecycle. **Handoff:** one controller for UI/CLI, with reviewer sign-off before on-board use.

First assignment

Start S1 and the first S2 rule using hand-checked examples. Do not broaden the DSL before hardware/evaluator agreement exists. The fake backend is a deliverable, not disposable throwaway code: it remains useful for UI regression and controller failure injection.

16. Delivery gates and demonstration acceptance

16.1 Integration gates

Gate	Required evidence
G0 — Independent development	Same working toolchain; checked contracts/fixtures; mock console; first evaluator tests; golden order decoding.
G1 — Hardware boundaries	Synthetic candidate transmitted by FPGA and decoded on host; identity/scratch register access works.
G2 — Complete data path	Ethernet replay reaches parser, book, generated rule, fixed checks, and outbound order for a deterministic scenario.
G3 — Operator workflow	TUI consumes real events, applies a reviewed parameter change, and shows a subsequent decision under the acknowledged new version.
G4 — Extensions	Assistant proposals, voice dictation, additional rules/slots, and polish added without regressing G3.

G3 is the required project demonstration. G4 is staged scope, not a prerequisite. Gates are dependency milestones rather than dates; event duration and final deadlines have not been specified.

16.2 Required P0 demonstration script

1. Start from a known reset/replay state. Display hardware mode, actual build identity, disarmed state, and manifest compatibility.
2. Initialize the supported book, review a profile, apply it through the controller, and show the acknowledged active configuration version.
3. Replay a no-signal case and a matching case. Show the correlated candidate, admission result, transmitted order, and host receipt.
4. Inspect a historical decision by ID and show its recorded snapshot, parameters, rule, and configuration version.
5. Apply a changed parameter using disable/idle/write/verify/activate. Replay the same scenario and show the expected changed behavior.
6. Demonstrate a blocked/suppressed candidate or disarmed case with a clear reason. Do not claim cancellation of an already sent order.
7. Show test summaries, clock/timing information, defined latency measurements, and any trace losses or limitations.

A P1 demonstration can begin with a typed or spoken request, validate/test it, and open a proposal for manual review. Keep a typed/manual path and prepared replay/profile available. Do not depend on an unplanned live synthesis run or uninterrupted cloud access.

16.3 Definition of done

A P0 package is done only when its acceptance test is reproducible, failure behavior is exercised, interfaces and provenance are documented, and the assigned reviewer has checked the cross-boundary behavior. A screenshot alone is insufficient.

The final bundle should contain source/build instructions, pinned dependencies, matching image/manifest, replay/profile fixtures, test results, a demo runbook, and known limitations. Separate hardware evidence from simulation and mock outputs. Claims about output rate, latency, loss, resource use, and supported semantics must be supported by the measured/tested configuration.

17. Extension tickets and CLI-agent working agreement

17.1 Removable P1 tickets

X1 — Assistant backend / Beginner B. Connect one provider to the existing bounded tools. Deliver typed request -> structured proposal -> validation/test evidence. Test unsupported rules, tool errors, ambiguous requests, stale context, and absence of deployment authority. Depends on S2-S4; may develop against fixtures first.

X2 — Assistant event integration / Beginner A. Connect U5 to X1, preserving conversation context, cancellation, proposal identity, and ordinary controls during requests. Test delayed/failed tools and invalid proposal links. This ticket connects an existing panel; it does not build a second agent framework.

X3 — Dictation / Beginner A. Record, stop, transcribe through ElevenLabs, and insert editable text into the draft. Test denial/missing microphone, timeout, cancellation, and explicit send. Depends on stable U5/X2 input behavior. No automatic submission or activation and no speech-output requirement.

A second example rule or additional compiled slots needs a separately reviewed ticket with arbitration, resource, and test implications. It is not implicit in X1-X3.

17.2 Instructions for implementation agents

Read this document, the assigned ticket, current shared interfaces, and the matching fixtures before editing. Treat P0 as required, P1 as opt-in, and P2 as excluded. Do not infer that an illustrative command, package, register address, or API already exists.

Work within the ticket boundary. Request a documented decision for unresolved shared semantics rather than choosing hidden widths, feed boundaries, packet layouts, or failure policies. Preserve module separation and use the existing fixtures/controllers instead of duplicating them.

Do not add a softcore, general interpreter, web frontend, real exchange connection, extra transport, or unrestricted AI tool access to simplify a local task. Do not perform hardware activation/programming without the designated operator workflow. Keep credentials and private recordings out of source control.

Finish with the actual files changed, build/test commands executed, results, remaining limitations, and any proposed contract changes. Never label an unrun test as passing or mock output as hardware evidence. Update documentation in the same change when an approved contract changes.

17.3 Proposed CLI surface

```
tickweave capabilities --json
tickweave validate <rule-or-proposal> --json
tickweave simulate <rule> --dataset <fixture> --json
tickweave plan <proposal> --target connected --json
tickweave inspect --run <run-id> --decision <id> --json
tickweave console --mode mock|simulation|hardware
```

These names are an interface proposal, not working commands. Select final syntax in the repository and keep machine-readable schemas, meaningful exit statuses, and normal human output consistent. GUI/TUI, CLI, and assistant operations should call the same libraries. Structural builds and operator-approved apply are distinct operations; normal file saves do not deploy.

18. Open decisions, risks, and change control

18.1 Resolve before the indicated gate

ID	Open item	Owner / deadline
O1	Exact Arty model, allowed demo hardware, pin/clock constraints, resource budget, and usable Ethernet TX path. No organizer restriction has been verified here.	Lead + experienced / G1
O2	Feed/schema subset, packet fixtures, depth, level indexing, initialization, sequence checks, and commit boundary.	Lead / C0 for H1-H2
O3	Field widths/signedness, arithmetic/overflow policy, supported operators, trigger behavior, and ID wrap/reset mapping.	Lead, reviewed with S2 / C0
O4	Structured interchange format, manifest/event schemas, CLI naming, and register-map generation approach.	Lead + Beginner B / C0
O5	Demo order bytes, addresses/ports, transmit ownership, buffer limits, and congestion policy.	Experienced + lead / E1-E2
O6	UART framing/rate/CRC/IDs; register layout; activation transaction semantics; or explicit JTAG substitution.	Experienced + lead / E3-H4
O7	Trace fields/size/queue/drop policy, host retention, replay pacing, and latency instrumentation endpoints.	Lead + experienced / G2
O8	Pinned OCaml/OxCaml/Dune/Bonsai dependencies, terminal minimum size, and reproducible setup.	Lead + Beginner A / G0
O9	Model provider/model, tool schema, credentials/limits, and ElevenLabs capture/privacy configuration.	Software/UI owners / before G4
O10	Named team assignments, repository/license, event duration, submission rules, and final deadlines.	Lead / project setup

18.2 Main delivery risks

Integration overload: the lead owns the hardest semantics. Mitigate with C0 contracts, independent harnesses, and G1 early. **Toolchain friction:** prove one Bonsai example in the shared environment before UI expansion. **Feed ambiguity:** freeze verified fixtures and update semantics rather than testing only successful inserts.

Transport/timing surprises: test TX, register access, stalls, and traces separately; keep replay rates modest until measured. **Scope growth:** keep AI/voice removable. **Ambiguous state:** rely on hardware acknowledgment and preserve unknown outcomes. **Cloud failure:** retain manual/typed configuration and deterministic fixtures.

18.3 Change process

Record the reason, affected requirement/ticket/schema, compatibility impact, test updates, and decision owner for each scope or interface change. Update this Markdown, increment its document version, and regenerate the proposal pair. Build IDs, schema versions, runtime configuration versions, and document versions serve different purposes; do not reuse one as another.

Change log: v0.1, October 2, 2026 — initial consolidation of the tickweave planning discussion, required baseline, staged features, team packages, and acceptance gates. Implementation remains unverified except for the lead's reported existing parser/networking assets.

19. Implementation references

These are primary-source implementation references checked on October 2, 2026. They support tooling descriptions and example pointers, not the project-specific requirements or a promise of compatibility. Pin tested revisions during G0. The project's scope and ownership derive from the planning discussion.

Core hardware and terminal stack

[1] **Jane Street — Hardcaml.** Circuit construction, simulation, and RTL generation reference.

<https://github.com/janestreet/hardcaml>

[2] **Jane Street — Bonsai Term.** Terminal application programming model and current setup instructions, including OxCaml.

https://github.com/janestreet/bonsai_term

[3] **Jane Street — Bonsai Term Examples.** Concrete layout, event, scrolling, text-input, and typography examples used as implementation references in U1-U5.

https://github.com/janestreet/bonsai_term_examples

[4] **Jane Street — Bonsai Term Components.** Reusable terminal components; verify exact installed library names and APIs against the pinned revision.

https://github.com/janestreet/bonsai_term_components

[5] **Jane Street — Bonsai Term Test.** Expect-test support for terminal application behavior.

https://github.com/janestreet/bonsai_term_test

[6] **aristocratos — btop README.** Visual/interaction reference for a compact operator console, including selection, filtering, sorting, details, and list pausing. No requirement to reuse btop code or assets.

<https://github.com/aristocratos/btop>

Optional assistant and transcription

[7] **OpenAI — Function calling.** Reference for the model-request/application-execution tool loop. This is a concrete implementation reference, not a locked provider choice.

<https://developers.openai.com/api/docs/guides/function-calling>

[8] **ElevenLabs — Create transcript.** Audio-file speech-to-text endpoint for record-then-transcribe input. No specific model ID, quota, or price is fixed by this document.

<https://elevenlabs.io/docs/api-reference/speech-to-text/convert>

End-state reminder

The assistant proposes and tests. The OCaml system validates and plans. The operator approves. The controller applies. The FPGA executes. The console shows what actually happened.